# VUE.JS AXIOS



## AJAX: ASYNCHRONOUS JAVASCRIPT AND XML

AJAX, Asynchronous JavaScript and XML, è un insieme di tecnologie per aggiornamenti incrementali nelle pagine web. Contiene:

- HTML/XHTML/CSS
- JavaScript
- DOM
- XML/XSLT
- XMLHttpRequest (a.k.a. XHR)



## XMLHTTPREQUEST

XMLHttpRequest è un oggetto per eseguire richieste web e ottenerne il risultato. Nonostante il suo nome, può essere utilizzato anche per richieste/risposte non-XML.

È supportato da ogni browser! Anche Internet Explorer lo supporta*! Se usi JavaScript puro (vanilla) e vuoi AJAX, è probabile che tu stia usando XMLHttpRequest.

* *sebbene Internet Explorer supporti XMLHttpRequest tramite ActiveX (**ugh**)*



## ESEMPIO DI UTILIZZO

```
let httpRequest;

function makeRequest() {
  httpRequest = new XMLHttpRequest();

  if (!httpRequest) {
    // alert("Giving up: Cannot create an XMLHTTP instance");
    return false;
  }

  httpRequest.onreadystatechange = alertContents;
  httpRequest.open("GET", "test.html");
  httpRequest.send();
}

function alertContents() {
  if (httpRequest.readyState === XMLHttpRequest.DONE) {
    if (httpRequest.status == 200) {
      // alert(httpRequest.responseText);
    } else {
      // alert("There was a problem with the request.");
    }
```

```
  }
}
```

Non useremo HXR direttamente, perché è troppo verboso. La sua interfaccia nor è aggiornata (es. manca supporto per js async).

- [ ]

Non useremo neanche le "Fetch API", un nuovo modo per ottenere dati da un server remoto usato dalle nuove applicazioni js.

Fetch è ancora un work-in-progress; es. mancano le seguenti feature:

- **timeout**
- gestione del progresso di upload/download
- backward compatibility



## AXIOS

un client HTTP per browser e node.js, basato sulle **Promise**.

E' un middleware



- **Richieste da Browser:** Esegue richieste **XMLHttpRequests** direttamente dal browser.
- **Basato sulle Promise:** Supporta completamente l'API **Promise**.
- **Interceptor:** Intercetta le richieste e le risposte per aggiungere logiche personalizzate.
- **Trasformazione dei Dati:** Trasforma automaticamente i dati di richiesta e risposta.
- **Annullamento delle Richieste:** Annulla le richieste utilizzando i meccanismi integrati.
- **Gestione Automatica del JSON:** Serializza e analizza automaticamente i dati **JSON**.
- **Serializzazione dei Form:** Serializza automaticamente gli oggetti di dati nei formati **multipart/form-data** o **x-www-form-urlencoded**.

- **Protezione XSRF:** Supporto lato client per la protezione da **Cross-Site Request Forgery**.

AXIOS IN VUE.JS / FANTASTIC COFFEE

WASAText\webui\src\main.js

```
// main.js
import {createApp} from 'vue'
import App from './App.vue'
import axios from 'axios'

let app = createApp(App);

// Aggiunge axios come proprietà globale, accessibile tramite this.$axios
app.config.globalProperties.$axios = axios;

app.mount('#app');
```

- [ ] Ora puoi usare `this.$axios` in ogni componente Vue.js

## Esempio GET:

```
// webui/src/views/HomeView.vue
async refresh() {
    try {
        let response = await this.$axios.get("/");
        // response.data contains the JSON in response body
    } catch (e) {
        console.error(e.toString());
    }
}
```



## Esempio POST:

```
async sendData() {
  try {
    let response = await this.$axios.post("/some/api", {
      firstName: 'Fred',
      lastName: 'Flintstone',
    });
    // response.data contiene il JSON nel corpo della risposta
  } catch (e) {
    console.error(e.toString());
  }
}
```



## DATI DELLA RISPOSTA

Un oggetto di risposta tipico da Axios contiene i seguenti campi:

```
{
  "data": {},
  "status": 200,
  "statusText": "OK",
  "headers": {},
  "config": {},
  "request": {}
}
```

## INTERCEPTOR DI RICHIESTA

Gli Interceptor permettono di intercettare e modificare le richieste o le risposte prima che vengano gestite dal codice `then` o `catch`.

```
axios.interceptors.request.use(function (config) {
  // Esegui qui modifiche alla configurazione della richiesta
  // (es. aggiungere header di autenticazione)
  return config;
}, function (error) {
  // Gestisci errori di richiesta (prima che la richiesta venga inviata)
  return Promise.reject(error);
});
```

Per il progetto, ci dobbiamo gestire l'invio del bearer token

## INTERCEPTOR DI RISPOSTA

```
axios.interceptors.response.use(function (response) {
  // Esegui qui modifiche o log della risposta riuscita
  return response;
}, function (error) {
  // Gestisci errori di risposta (es. errori 4xx, 5xx)
  return Promise.reject(error);
});
```

# CANCELLAZIONE

Importante, ma non l'abbiamo visto bene a lezione

```
// 1. Crea un controller
const controller = new AbortController();

// 2. Utilizzo:
async function fetchWithCancellation() {
  try {
    const response = await this.$axios.get('/user/12345', {
      signal: controller.signal // Passa il signal alla richiesta
    });
  } catch (e) {
    if (axios.isCancel(e)) {
      console.log('Richiesta annullata', e.message);
    } else {
      // gestisci errore
    }
  }
}

// 3. Annulla la richiesta
controller.abort();
```

## LINK UTILI

- https://axios-http.com/
- https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest

